INEM · IA pour les systèmes embarqués · Réseaux de neurones

Comprendre le perceptron, du pixel à la prédiction

Reconnaître des chiffres en quatre étapes progressives

Travaux dirigés et initiation à la recherche · Roland Cédric TAYO

Résumé

Ce travail conduit du neurone simple au réseau multicouche. Les fonctions sont écrites à la main : lire les pixels, calculer une sortie, corriger les poids, puis tester. Chaque étape distingue les activités et les résultats à conserver. Une petite étude comparative apprend à formuler une hypothèse et à interpréter les mesures.

Mots-clés : perceptron ; classification ; apprentissage ; généralisation ; expérimentation.

Objectifs et parcours

Comprendre comment des nombres entrent dans un neurone, comment ses poids changent pendant l'apprentissage et comment vérifier ce qu'il a appris.

Lire des pixels → Calculer une sortie → Corriger les poids → Tester et tracer

LE PARCOURS

Étape et dossier de référence	Ce que l'on découvre
1. Généralisation `1-Generalisation`	Un neurone distingue 0 et 1 ; on perturbe ensuite les pixels pour tester sa résistance au bruit.
2. Deux classes `2-Separation-deux-classes`	La règle de Widrow–Hoff apprend à partir d'un score réel, au lieu de la seule décision 0 ou 1.
3. Dix classes `3-Separation-dix-classes`	Dix neurones donnent dix scores. Le score le plus grand désigne le chiffre prédit.
4. Multicouche `4-Perceptron-Multicouches`	Une couche cachée et la rétropropagation permettent d'aborder les chiffres manuscrits MNIST.

Progression. Les étapes 1 à 3 construisent les bases. L'étape 4 ouvre le prolongement : lire une image et comprendre les calculs avant de terminer l'entraînement. Commencer par vérifier le petit réseau avant de programmer l'ensemble sur MNIST.

RÈGLE DE TRAVAIL

Écrire les calculs à la main en C, avec la bibliothèque standard uniquement. Commenter, expliquer et tester chaque fonction. Les outils de figures sont autorisés. Les règles détaillées figurent page 7.

Prérequis : fonctions, boucles, tableaux et fichiers ; sommes de produits. Les dérivées et leur enchaînement sont nécessaires à l'étape 4.

ÉTAPE 1 · 1-GENERALISATION

Un neurone distingue les chiffres 0 et 1

Question. Peut-il reconnaître deux motifs, puis leurs versions abîmées ?

zero.txt

```
......
.****.
.*..*.
.*..*.
.*..*.
.*..*.
.****.
......
0
```

un.txt

```
......
...*..
..**..
...*..
...*..
...*..
..***.
......
1
```

Chaque fichier comporte **8 lignes de 6 caractères**, puis la classe en 9e ligne.

. devient 0 ; * devient 1.

Lire ligne par ligne : on obtient 48 entrées x_j, avec j de 0 à 47.

ACTIVITÉ — CONSTRUIRE, APPRENDRE, TESTER

1. Créer les deux fichiers. Écrire les fonctions de lecture, de calcul du score, de décision et de mise à jour. Initialiser les poids w_j dans [−0,1 ; 0,1], avec une graine enregistrée (par exemple 42). Initialiser le biais **b=−0,5** et fixer le pas **ε=0,01**.
2. Pour un exemple, calculer le score a, la prédiction ŷ et l'erreur e. Puis corriger les poids et le biais :

$$a = \Sigma_{j=0}^{47} w_j x_j + b \; ; \quad \hat{y} = 1 \text{ si } a \geq 0, \text{ sinon } 0.$$
$$e = d - \hat{y} \; ; \quad w_j \leftarrow w_j + \varepsilon \, e \, x_j \; ; \quad b \leftarrow b + \varepsilon \, e.$$

Lire la recette : d est la classe attendue ; ŷ est la classe prédite. Σ signifie additionner ; ← signifie remplacer par la nouvelle valeur. Le biais b décale la frontière de décision. Le seuil θ=0,5 du sujet initial correspond à b=−θ avant apprentissage.

3. Une époque présente les deux motifs, une fois chacun, dans un ordre mélangé. À la fin, **sans modifier les poids**, recompter les erreurs sur les deux motifs. Arrêter à zéro erreur, ou après 1 000 époques en signalant la limite.
4. Pour p=0, 10, …, 100 % de bruit, copier le motif et inverser exactement n=arrondi(48p/100) pixels **distincts**, tirés au hasard. Tester 50 copies par chiffre et par p. Garder les mêmes copies pour l'étape 2 ; ne jamais apprendre sur ces copies.

RÉSULTATS À CONSERVER

- Une figure à deux panneaux : erreurs sur les deux motifs selon l'époque ; taux d'erreur selon le bruit, avec une courbe par chiffre.
- CSV, graine et poids appris. Taux d'erreur (%) = 100 × nombre de mauvaises réponses / nombre d'essais.
- Quatre phrases : que change l'apprentissage ? Que se passe-t-il avec le bruit ? Ces tests ne prouvent pas la reconnaissance de toutes les écritures.

Sur papier : pour x=[1,0], w=[0,2 ; −0,1], b=−0,5 et d=1, calculer une sortie puis une mise à jour. Ce mini-exemple a deux entrées, pas 48.

ÉTAPE 2 · 2-SEPARATION-DEUX-CLASSES

Changer la règle d'apprentissage : Widrow–Hoff

Question. Que change une correction fondée sur la valeur du score, plutôt que sur la seule décision ?

Même motif x → Score réel a → Erreur d − a → Correction des poids

CE QUI CHANGE

Le score reste $a=\Sigma_j w_j x_j+b$. À l'étape 1, la correction utilisait $d-\hat{y}$: une décision déjà correcte ne produisait plus de correction. Ici, elle utilise **d−a** : le score continue à se rapprocher de la cible numérique 0 ou 1.

$$e = d - a \; ; \quad w_j \leftarrow w_j + \varepsilon \, e \, x_j \; ; \quad b \leftarrow b + \varepsilon \, e.$$

Pour classer après calcul : $\hat{y} = 1$ si $a \geq 0{,}5$, sinon 0.

Le score réel n'est pas limité à [0 ; 1] et n'est pas une probabilité. Le seuil de décision vaut ici 0,5 parce que les cibles numériques valent 0 et 1. Il ne remplace pas le biais b.

ACTIVITÉ — MODIFIER UNE CHOSE ET COMPARER

1. Repartir des **poids initiaux** de l'étape 1, pas de ses poids appris. Conserver les motifs, le biais initial et $\varepsilon=0{,}01$. Remplacer seulement la règle d'erreur et utiliser le seuil de décision indiqué ci-dessus.
2. Sur un petit exemple, calculer à la main la correction avec $d-\hat{y}$, puis avec d−a. Écrire ce qui diffère. Dans le programme, distinguer explicitement `score`, `prediction` et `target`.
3. Présenter les deux motifs à chaque époque. Après l'époque, calculer la perte L sur les deux motifs avec des poids figés. Arrêter si $L \leq 10^{-4}$, ou après 1 000 époques. Si elle explose ou devient non finie, vérifier la mise à jour ; refaire ensuite un essai à $\varepsilon=0{,}001$ en le signalant.
4. Tester les **mêmes copies bruitées** qu'à l'étape 1, sans mise à jour. Comparer la robustesse des deux règles.

$$L = (1 / 2N) \, \Sigma_{k=1}^{N} \, (d_k - a_k)^2 \; ; \quad \text{ici } N=2.$$

N est le nombre de motifs évalués ; k désigne un motif ; d_k est sa cible et a_k son score. Le carré pénalise l'écart numérique. Le facteur ½ simplifie la dérivée. Une perte numérique et un taux de mauvaises classes sont deux mesures différentes.

RÉSULTATS À CONSERVER

- Une figure : perte L selon l'époque et comparaison des courbes d'erreur au bruit des deux méthodes. Nommer les axes et les courbes.
- Un tableau donnant, pour chaque méthode, les scores et décisions sur les deux motifs propres, puis l'erreur à 10 %, 30 % et 50 % de bruit.
- Quatre phrases : quelle règle fonctionne le mieux dans votre essai, pour quel chiffre et dans quelles limites ?

À examiner, pas à supposer. Widrow–Hoff réduit une erreur quadratique. Il ne garantit ni un écart égal aux deux classes, ni des courbes de bruit symétriques, ni une meilleure robustesse dans tous les cas.

ÉTAPE 3 · 3-SEPARATION-DIX-CLASSES

Passer de deux chiffres à dix chiffres

Question. Comment choisir une classe parmi dix, au lieu de répondre seulement 0 ou 1 ?

48 pixels → 10 neurones linéaires → 10 scores → Le plus grand gagne

ACTIVITÉ — ÉTENDRE LE PROGRAMME

1. Créer les dix fichiers : `zero.txt`, `un.txt`, `deux.txt`, `trois.txt`, `quatre.txt`, `cinq.txt`, `six.txt`, `sept.txt`, `huit.txt` et `neuf.txt`. Même format : 8 lignes de 6 pixels, puis la classe de 0 à 9.
2. Écrire le vecteur cible d de longueur 10 : une seule case vaut 1, celle du chiffre attendu ; les autres valent 0. Par exemple, pour le chiffre 2 : d=[0,0,1,0,0,0,0,0,0,0].
3. Créer W, tableau de **10 lignes et 48 colonnes**, et 10 biais b_i. W_{ij} relie le pixel j à la sortie i. Initialiser les poids comme avant et les biais à zéro ; fixer ε=0,01.
4. À partir d'un motif, calculer les dix scores puis appliquer Widrow–Hoff à chaque sortie :

$$a_i = \Sigma_{j=0}^{47} W_{ij}x_j + b_i \; ; \quad e_i = d_i - a_i,$$
$$W_{ij} \leftarrow W_{ij} + \varepsilon\, e_i x_j \; ; \quad b_i \leftarrow b_i + \varepsilon\, e_i.$$

i va de 0 à 9 ; j de 0 à 47. Calculer les dix erreurs à partir des scores avant mise à jour. Pour prédire, choisir $\mathbf{argmax_i}\ \mathbf{a_i}$: l'indice du score le plus grand. En cas d'égalité, prendre le plus petit indice. Ne pas chercher dix décisions binaires indépendantes.

5. Une époque parcourt les dix motifs dans un ordre mélangé. Calculer ensuite, sans apprendre, le nombre de classes correctes et $L=(1/2N)\Sigma_k\Sigma_i(d_{ki}-a_{ki})^2$, avec N=10. Arrêter si $L \le 10^{-4}$, ou après 1 000 époques ; noter le résultat réellement obtenu.
6. Pour chaque chiffre, produire 50 variantes à 10 % de bruit, sur des copies séparées. Remplir une matrice de confusion 10×10 : **ligne = chiffre réel ; colonne = chiffre prédit**. Chaque ligne contient donc 50 essais.

RÉSULTATS À CONSERVER

- Une figure à deux panneaux : taux de reconnaissance sur les dix motifs selon l'époque ; matrice de confusion sur les 500 variantes bruitées.
- Un tableau des dix prédictions sur motifs propres et le taux global sur les variantes : 100 × bonnes réponses / 500.
- Quatre phrases : quels chiffres se confondent ? Dix motifs appris ne représentent pas toutes les façons d'écrire les chiffres.

Contrôle. Vous devez avoir 480 poids et 10 biais. Chaque motif donne exactement une classe prédite. Les poids restent figés pendant les évaluations.

ÉTAPE 4 · 4-PERCEPTRON-MULTICOUCHES

Une couche cachée pour les chiffres manuscrits

Question. Comment apprendre des transformations intermédiaires au lieu de relier directement chaque pixel aux sorties ? Les données et leur lecture sont décrites page suivante.

MNIST : 28 × 28 → 784 entrées → 100 neurones cachés → 10 sorties

ACTIVITÉ — SUIVRE LES CALCULS, PUIS PROGRAMMER

1. Afficher une image MNIST et son étiquette. Convertir chaque pixel p_j en $x_j = p_j/255$: les entrées sont entre 0 et 1. Créer la cible d de dix valeurs comme à l'étape 3.
2. Créer $W^{(1)}$ de taille 100×784, $W^{(2)}$ de taille 10×100 et les biais $b^{(1)}$, $b^{(2)}$. Initialiser les biais à zéro ; tirer chaque poids entre $-1/\sqrt{n}$ et $+1/\sqrt{n}$, où n est le nombre d'entrées de sa couche (784 ou 100). Garder la graine.
3. Calculer d'abord la couche cachée, puis les sorties, avec la sigmoïde σ :

$$\sigma(u) = 1/(1+\exp(-u)) \; ; \quad \sigma'(u) = \sigma(u)[1-\sigma(u)].$$
$$u_h = \Sigma_j W^{(1)}_{hj} x_j + b^{(1)}_h \; ; \quad z_h = \sigma(u_h),$$
$$v_i = \Sigma_h W^{(2)}_{ih} z_h + b^{(2)}_i \; ; \quad s_i = \sigma(v_i).$$

h désigne un neurone caché (0 à 99), j un pixel (0 à 783), i une sortie (0 à 9). z est une activation cachée ; s est la sortie. exp est l'exponentielle ; σ′ est la dérivée. Les exposants (1) et (2) désignent les couches, pas des puissances.

4. Pour la perte d'un exemple $L = \tfrac{1}{2}\Sigma_i (d_i - s_i)^2$, calculer les signaux de correction de la sortie, puis de la couche cachée :

$$\delta_i = (d_i - s_i) s_i (1 - s_i),$$
$$\delta_h = z_h (1 - z_h) \, \Sigma_i W^{(2)}_{ih} \delta_i.$$

5. Quand **tous les δ sont calculés avec les anciens poids**, mettre à jour : $W^{(2)}_{ih} \leftarrow W^{(2)}_{ih} + \varepsilon \delta_i z_h$; $W^{(1)}_{hj} \leftarrow W^{(1)}_{hj} + \varepsilon \delta_h x_j$. Ajouter εδ au biais correspondant. Ici δ est l'opposé du gradient local : le signe + est cohérent.
6. Vérifier d'abord sur papier un réseau réduit 2→2→2, puis la même sortie dans le programme. Entraîner sur MNIST selon le protocole page 6. Prédire avec $\text{argmax}_i s_i$, sans modifier les poids au test.

RÉSULTATS À CONSERVER

- Pour amorcer : une image avec son label, les dimensions vérifiées et la trace du petit calcul 2→2→2.
- Pour terminer : une figure des pertes train/validation selon l'époque ; un tableau de reconnaissance sur le test et dix exemples réel/prédit.
- Une explication : comment l'erreur de sortie corrige-t-elle aussi la couche cachée ? Aucun score de 96 % n'est imposé ou garanti.

FICHE PRATIQUE · DONNÉES, VÉRIFICATIONS ET REMISE

Lire MNIST et terminer sans ambiguïté

1. LES QUATRE FICHIERS

Le miroir de la CVDF propose les fichiers compressés et décrit le format IDX. Télécharger puis décompresser les quatre fichiers `.gz`. Si les données du dossier de travail sont déjà disponibles, vérifier leurs en-têtes avant utilisation.

Fichier décompressé	Contenu et en-tête à lire
`train-images-idx3-ubyte`	60 000 images 28×28 ; 16 octets d'en-tête.
`train-labels-idx1-ubyte`	60 000 labels de 0 à 9 ; 8 octets d'en-tête.
`t10k-images-idx3-ubyte`	10 000 images de test ; 16 octets d'en-tête.
`t10k-labels-idx1-ubyte`	10 000 labels de test ; 8 octets d'en-tête.

Les nombres d'en-tête sont des entiers de 32 bits en **big-endian** : l'octet de poids fort vient en premier. Lire type, nombre d'éléments et dimensions. Une image occupe ensuite 784 octets non signés. Pour l'indice r, lire l'image à 16+784r et le label à 8+r ; les indices commencent à 0. Ainsi, les indices d'entraînement vont de 0 à 59 999.

2. PREMIÈRE EXÉCUTION DE L'ÉTAPE 4

1. Prendre les images train d'indices 0 à 4 999 pour apprendre et 5 000 à 5 999 pour valider. Ces ensembles ne se recouvrent pas. Les fichiers officiels de test restent réservés à l'évaluation finale.
2. Utiliser ε=0,01, une mise à jour par exemple et au plus cinq époques. Mélanger les exemples train à chaque époque ; chronométrer la première. À chaque fin d'époque, calculer perte et taux de reconnaissance sur train et validation, **sans mise à jour**. Garder les poids de plus faible perte de validation.
3. Évaluer une fois ces poids sur les 10 000 images de test. Si vous commencez par 1 000 pour vérifier le programme, appeler ce résultat « test partiel, N=1 000 ». Ne pas ajuster les réglages à partir du test.

Ce démarrage réduit sert à comprendre et déboguer. L'utilisation d'un plus grand ensemble d'apprentissage est un prolongement, avec une nouvelle partition documentée. Une baisse de perte ne garantit pas à elle seule une bonne reconnaissance sur le test.

3. VÉRIFIER ET CONSERVER

Pour chaque étape : sources commentées, tests, réglages, graines, poids, CSV, figure et analyse. Distinguer ce qui est terminé de ce qui est amorcé. L'étude comparative et la présentation éventuelle sont précisées page 7.

En cas de blocage : vérifier d'abord lecture et dimensions ; puis normalisation, signes et ordre des mises à jour. Revenir au petit calcul avant de relancer un entraînement.

Sujet initial : « Reconnaissance de caractères », Pierre Andry, Université de Cergy-Pontoise (`toDo.pdf`). Adaptation pédagogique : Roland Cédric TAYO.

Lectures : CVDF — MNIST et format IDX ; Stanford CS231n — règle de chaîne et rétropropagation. Ces lectures aident à comprendre ; elles ne remplacent pas vos calculs.

PETITE RECHERCHE · COMPRÉHENSION ET RÉSULTATS

Comparer une idée, puis expliquer ce que l'on a appris

Cette étude prolonge les étapes 1 et 2. Elle porte sur votre propre programme et reste limitée à une modification.

1. PETITE RECHERCHE — UNE MODIFICATION, UNE COMPARAISON

Sur l'étape 1 ou 2 : **que change ε=0,001 au lieu de 0,01 ?** Écrire d'abord une hypothèse sur les époques nécessaires et l'erreur à 30 % de bruit.

Comparer les deux pas pour les graines 42, 43 et 44 : six essais au total. Pour chaque paire, garder **poids initiaux, ordre des exemples et motifs bruités identiques**. Conserver les critères d'arrêt et le plafond de 1 000 époques.

Résultats : six lignes CSV (graine, pas, époques, arrêt atteint ou non, erreur propre et à 30 % de bruit), une figure montrant tous les essais, puis une conclusion et un prochain test. Un résultat nul ou défavorable est recevable ; atteindre le plafond n'est pas converger.

2. PRÉSENTATION ÉVENTUELLE

Préparer : une fonction et son calcul ; les résultats et les courbes ; la recherche ; les limites et la suite. Chacun peut être interrogé. Distinguer étapes terminées et amorcées.

Remise : code commenté, tests, réglages, graines, poids, CSV, figures et analyse. Taux de reconnaissance =100×bonnes classes/N ; taux d'erreur =100−ce taux. Une somme de δ n'est pas un taux d'erreur.

OUTILS ET MÉTHODE

- Un PC, un compilateur **C** et un outil de figures : Gnuplot ou Python/Matplotlib. **Concevoir et écrire à la main** score, décision, perte, mise à jour, gradients et entraînement. Ne pas recopier les anciens programmes.
- Aucune bibliothèque de calcul matriciel ou d'IA, aucun framework ni autodifférentiation. La bibliothèque standard C reste autorisée : fichiers, mémoire, aléatoire, `exp`, `sqrt`. Les outils de figures sont autorisés.
- **Documenter chaque fonction :** rôle, entrées et dimensions, sortie, équation, commentaire des choix et petit test. Code et commentaires en anglais de préférence ; chacun doit pouvoir expliquer ses fonctions.
- **Papier → fonction → test → figure → explication.** Garder CSV et réglages. Recherche et présentation éventuelle : page 7.